

TypeScript ライブラリとして動作する組版エンジン minitype を公開しました

2025 年度下期未踏アドバンス事業¹にて開発してきた、TypeScript ライブラリとして動作する組版エンジン **minitype** を公開しました！ 従来は独立したシステムや専用言語として提供されることの多かった組版処理をライブラリとして提供することにより、LLM や外部アプリケーションを含む、様々な文書生成に活用されることを目指しています。

・未踏アドバンス事業：2025 年度下期実施プロジェクト概要（和田 PJ） | デジタル人材の育成 | IPA 独立行政法人 情報処理推進機構

minitype の使用方法や活用例のデモは、以下の動画に端的にまとまっています。百聞は一見に如かずとも言いますので、まずはこちらをご覧くださいいただければと思います。

本記事では、minitype の開発内容、開発背景、使用方法、活用例（ユースケース）等を、サンプルコードや出力例も交えながらご紹介していきます。なお、本記事は PDF 文書としてもご覧いただけます！（minitype による組版です）

なにをつくったか

TypeScript ライブラリとして動作する組版エンジンを開発しました。分かりやすく換言すると、大規模言語モデル（LLM）や他のアプリケーションから美しく整形された文書を作成するために、**高品質な組版処理を通常の TypeScript プログラムと同じように呼び出せる仕組み**を作った！ということになります（図 1）。

GUI を持たず、プログラムからの操作を前提とするブラウザ（例：Playwright, Puppeteer）はヘッドレスブラウザと呼ばれますが、これになぞらえて minitype では、組版処理を固有の組版言語（DSL）や GUI から切り離して、汎用プログラミング言語から直接実行可能な組版エンジンを「**ヘッドレス組版エンジン**」と定義しています。

プロジェクト概要

ヘッドレス組版エンジン minitype

アプリケーションや AI から美しい文書を生成する技術

→ 多様な文書生成ワークフローに高品質な組版処理を組込む

組版処理：文字や図表等の要素を適切に配置して、読みやすい文書に仕上げる処理



図 1：minitype により作成可能な文書の例。

minitype の特徴

minitype は大きく次の特徴を有します。これらの特徴により、特定のアプリケーションや専用言語の中に閉じることなく、TypeScript/JavaScript のエコシステムから高度に組版された文書を柔軟に生成できるように設計されています。

- ・ **npm ライブラリ**（パッケージ）として提供され、JavaScript のランタイム上に簡単にインストール可能
 - ・ 複数の **JavaScript ランタイム**（Node.js, Bun, Web ブラウザ）で動作
- ・ **TypeScript オブジェクトや関数**によって文書構造やレイアウトを記述
- ・ ライブラリが提供する API を呼び出すことにより組版処理を実行²
- ・ **高度な組版処理**に対応（日本語組版処理の要件や

¹ 情報処理推進機構（IPA）の実施する、ソフトウェア開発等に対する公的な支援事業。

² minitype では行分割処理やページ処理等に注力することにして、OpenType や PDF の描画処理に関しては opentype.js, pdf-lib, および PDFKit を使用しています。

Knuth–Plass Line-Breaking Algorithm をベースとする)

- PDF 生成のみならず、組版結果のレイアウトや診断情報を取得可能

構成要素

minitype では、以下の 3 つの TypeScript パッケージを提供しています。

- **@minitype/minitype**：組版処理を担当するコアエンジン
- **create-minitype**：CLI または API 経由で minitype を用いた文書プロジェクトを簡単にセットアップするツール
- **@minitype/vite-plugin**：文書のリアルタイムプレビューを実現する Vite プラグイン

これらのパッケージは npm 上で配布されています。minitype 本体については PolyForm Noncommercial License 1.0.0 としつつも、個人利用や同人利用に関しては自由に使用していただくことが可能です（商用利用についてはお問い合わせください）。また、create-minitype、vite-plugin についてはオープンソース（MIT ライセンス）で公開しています。

開発背景

ここで、minitype を開発した背景として、そもそも組版とは何か？という話と、変化しつつある文書生成について説明します。

文書生成と組版の広がり

我々は情報を残したり、その情報を共有したりするために、日々様々な文書を作成しています。ここでいう文書には、レポートや論文、書籍に限らず、申請書、請求書、マニュアル、スライド等、情報を一定の形式に整理した幅広い媒体が含まれます。特に、PDF を中心とした固定の文書フォーマットは、情報の保存性や可搬性、読みやすさに優れることから、Web が普及した現在でも依然として頻繁に使用されています。

ここで重要となるのが**組版**です。組版とは、情報を読みやすく整えるために、文字や図版等の要素を適切に配置する

背景 (1/3) 文書生成の広がり

情報伝達を支える文書、その作られ方は変わりつつある
レポート、論文、申請書、請求書、マニュアル……

手動での文書作成

人が文章を書き、Word や InDesign で見た目を整える

AI エージェントの活用

LLM (ChatGPT 等) が執筆したレポートを PDF 文書として出力

アプリケーションへの組込

UGC (ブログ記事等) やデータベースから文書を自動生成

図 2：文書生成の広がり。

作業を指します。文書中のレイアウトやフォントは情報の理解のしやすさや信頼性に影響を与えると報告されている（論文 1, 論文 2）ことから、情報を正確かつ効果的に伝える上で、適切な組版は欠かせない要素です。

また現在では、こうした文書の作られ方も変化しつつあります。これまでのように、人が文章を書き、見た目を整えて文書を作るだけでなく、**LLM が文書を生成したり³、アプリケーションから大量の文書を自動生成したりする**機会は格段に増加しています（図 2）。

しかしながら、ChatGPT 等の生成 AI を用いて文書を生成すると、見た目が単調になったり、画像生成を介した場合には文字の細部が潰れたりすることがあります（図 3 左）。また、LaTeX や Typst 等の既存の組版システムは、**専用の組版言語を中心に、モノリシックなソフトウェアとして設計されているため、通常のプログラムの一部として組版処理を実行することには主眼を置いていません**。そのため、LLM や外部アプリケーションから組版処理を実行しようとする、TeX/Typst 言語によるソースコードの生成や、外部プロセスの呼び出しを介する必要があります（図 3 右）。

こうした背景を踏まえて、**LLM や外部アプリケーションとの連携を前提**としつつも、高品質な組版処理を柔軟に実行できる仕組みが求められています。

日本語組版の特異性

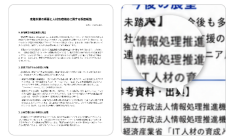
さらに、我々が普段から扱う日本語には、縦組、禁則処理、ルビ、異体字⁴といった、**言語に固有かつ複雑な組版要件⁵**が数多く存在しています（図 4）。こうした処理は海外由来のソフトウェアでは十分に対応されないことも多く、

³ ちなみに本記事の文章はすべて人手で書いています。このようなことを書かないといけない世の中になってしまいました。

⁴ 漢字に複数のバリエーションがあること。扱うには Unicode に備わった特殊な仕組みが必要。

背景 (2/3) 文書生成の課題

LLM による文書生成



従来の組版システムを
活用しようにも……

例: LaTeX, Typst, InDesign

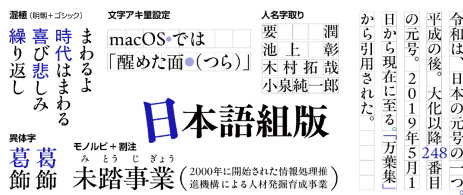
- ・ LLM や外部アプリケーションとの連携が難しい
- ・ 専用の組版言語の習得が必要

→ LLM や外部アプリケーションと連携しつつ
高品質な組版処理を行うための仕組みが必要

5

図 3: 文書生成における課題。LLM による文書も改善傾向にあるものの、依然として「AI っぽさ」を感じることは多いように思います。

背景 (3/3) 日本語組版の複雑さ



→ 日本語組版技術を国内で開発・維持することが重要

6

図 4: 日本語組版要件を紹介する際に度々使用しているスライド。

ゆえに日本語による文書生成を支える組版エンジンを、国内で開発することが重要であると考えています⁶。

そこで、LaTeX や Typst 等で実現されている本格的な日本語組版処理や文書作成機能を備えながらも、ライブラリという形で新たな組版エンジンを提供すれば、人手による文書作成に限らず、LLM やアプリケーションを介した新たな文書生成にも高品質な組版処理を組込むことが可能だと考えるに至りました。minitype は、特定のアプリケーションや組版言語に閉じることなく、**様々な文書生成ワークフローから利用できる基盤の組版エンジン**になることを目指しています。

⁵ 日本語組版処理の複雑さはアドビも認めるところであり、例えば InDesign 日本語版の開発にあたっては、現場との対話を繰り返しながら専用のテキストエンジンを開発したと述べられています (参考記事)。

⁶ 日本語以外にも、アラビア語やヒンディー語等、複雑な組版要件が存在する言語は数多く存在します。ここで主張したいのは、日本語だけが難しいというわけではなく、日本語のように世界的にはマイナーな言語は、欧文 (英語) 中心の世界での開発からは取り残されてしまう可能性があるということです。

minitype による文書作成

次に、minitype による文書作成の方法を見ていきます。minitype を実行する最小限のソースコードを以下に示します。@minitype/minitype をインストールしたうえで、以下のプログラムを `tsx index.ts` で実行すると、`hello.pdf` が生成されます。

```
// index.ts
import { h1, minitype, p } from
"@minitype/minitype";
await minitype([ {
  body: [
    h1("Hello World!"),
    p("minitype のテストです"),
  ],
} ]).save("hello.pdf");
```

minitype では、文書の内容やレイアウトを、TypeScript のオブジェクトや関数を組み合わせて宣言的に記述します。例えば、上述のソースコードでは、`h1` 関数で見出しを、`p` 関数で段落を表現した上で、これらの**ブロック要素**を配列として組み合わせることによって文書構造を記述します。作成した配列を `minitype` 関数に渡すと組版処理が実行されます。

見出しや段落に限らず、minitype はリスト、ソースコード、画像、表、数式⁷、脚注、相互参照等、**文書作成に必要な様々な要素**を標準で用意しています。また、図表や見出しの自動採番、目次、参考文献、Markdown からの変換等は**プラグイン**として実現されています。プラグインは頻繁な利用が見込まれるものを標準提供するほか、ユーザー自身が独自に作成することもできます。

レイアウト、スタイル

レイアウトについては、上から下へ要素を並べることに加えて、左右に分けて列を配置する**段組**、ブロックに対して装飾を加える**ボックス**、複数のブロックを横に並べる**フレックスボックス**等を実装しています。例えば、要素を左右に配置する場合、以下のように記述します。これらを組み合わせることにより、レポートのような一般的な文書だけでなく、後

⁷ MathJax によって生成された SVG を埋め込んでいます。ライブラリが TypeScript で書かれているため、豊富な Web 用ライブラリを活用できる点も minitype の利点の一つです。

ほど紹介するスライドやビジネスレポートのように、複数の要素を自由に配置した文書も作成することができます。

```
flexbox([
  box([p(" 左カラム ")], { inlineSize:
    ratio(0.3) }),
  box([p(" 右カラム ")])],
]);
```

スタイルに関しては、文書レベルでスタイルを一括指定できるほか、ページレベルでの処理 (group)、複数のブロックに対する一括指定 (section)、個別のブロックに対する指定が可能です。その他、ページレベルおよび行・文字レベルでの処理として、以下に対応しています。

- ・ **ページレベルでの処理** ブロック間のスペースの調整 (gap)、文書への共通要素の指定 (flow)、図版のフロート、垂直スペース、ページの背景画像、図形 (矩形/円) 描画、改ページ/改段、PDF しおり
- ・ **行レベルでの処理** 書字方向の制御 (横組/縦組)、禁則処理 (追出し、追込み、ぶらさがり処理)、分離・分割禁止処理、行取り、欧文組版、ドロップキャップ
- ・ **文字レベルでの処理** 文字色/フォント/文字サイズ/文字揃えの指定、カーニングの調整、合成フォント⁸、禁則処理、文字組アキ量設定⁹、タブ処理、均等割付、インライングラフィック (行中への画像挿入)、文中への数式挿入、縦中横、異体字セレクタ・CID・絵文字の挿入、下線、上線、打ち消し線、文字背景、PDF のハイパーリンク

他の処理と組み合わせて使用する

minitype は TypeScript ライブラリであるため、通常の Web 開発と同様に、**JavaScript の構文 (関数、変数、制御構造等) や API を利用して文書を構成**できます。例えば、配列から文書要素を生成する場合、map 関数を使用し以下のように記述できます。

```
const members = ["Alice", "Bob",
```

⁸ 漢字、かな、欧文等の複数の文字種に対して別々のフォントを割り当てる指定。

⁹ 文字と文字が並んだ際の間隔 (アキ) を調整する設定。句読点・括弧等の約物 (やくもの) や、和文と欧文とが並んだ際に特に重要となる。

```
"Carol"]];
const document = [
  h1("Members"),
  ...members.map((name) => p(name)),
];
```

繰り返し使用するレイアウトは React におけるコンポーネントのように、**ヘルパ関数**として切り出すことにより、記述を簡略化することができます。

```
const card = (title: string,
description: string) =>
  box([h2(title), p(description)]);
const document = [
  card("Tokyo", "3 nights"),
  card("Kyoto", "2 nights"),
];
```

さらに、Fetch API や既存の npm ライブラリ等と組み合わせて、取得・加工したデータをそのまま文書へ流し込むことも可能です。このように、**データの取得・処理・組版を一つの TypeScript プログラムの中で完結**させられる点が、ライブラリとして提供される minitype の大きな特徴になっています。

```
const response = await
fetch("https://example.com/api/users");
const users = await response.json();
const document = [
  h1("ユーザー一覧"),
  easytable([
    ["名前", "所属"],
    ...users.map((u) => [u.name,
u.organization]),
  ]),
];
```

スペースの都合ですべての仕様を紹介することはできませんが、minitype には他にも様々な機能があります。詳細な使用方法や仕様に関しては、**クイックスタート** や **リファレンス** を参照していただければと思います。



図 5: create-minitype の実行画面。

minitype を試してみる

ここからは、minitype の具体的な使用方法を示すとともに、minitype が従来の組版システムの域を超えて、様々な場面で活用可能であるというユースケースを紹介していきます。

手動での文書作成

まず、手動での文書作成を行う例をご紹介します。minitype 本体は TypeScript ライブラリとして提供されていますが、プロジェクトのセットアップを支援する create-minitype を使用することで、**文書生成を手軽に始められる**ようになっています。Node.js がインストールされた環境で以下のコマンドを実行すると、使用可能な文書のテンプレートが表示されます（図 5）。

```
npm create minitype # npm
yarn create minitype # Yarn
```

ここでは、学会投稿を想定したテンプレートである conference-paper を選択してみます。これにより、文書のビルドに必要なソースコード一式と、@minitype/minitype を含む依存パッケージがセットアップされます。あとは、作成されたディレクトリ配下で npm run build を実行することにより TypeScript プログラムが実行され、PDF 文書を生成することができます。

プロジェクトには Vite プラグイン (@minitype/vite-plugin) も併せてインストールされるため、npm run dev を実行することにより、リアルタイムプレビュー機能を提供するローカル Web サーバが起動します。

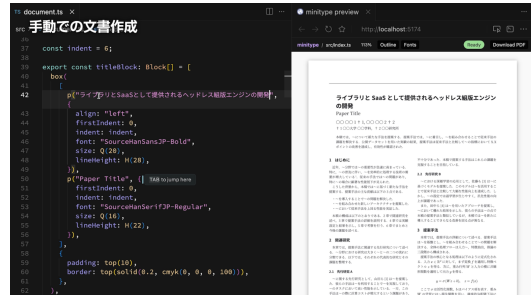


図 6: vite-plugin の起動画面。左側にソースコードが、右側にローカルサーバ上に起動したリアルタイムプレビュー画面が表示される。

プレビュー画面をエディタ内のブラウザに表示することにより、TypeScript のソースコードを編集する度に変更内容が反映され、**仕上がりを確認しながら文書を編集**することができます（図 6）。

create-minitype には、この他にもレポート、学位論文、ビジネスレポート、履歴書をはじめとする 10 種類のテンプレートを用意しています。もちろん、ユーザが 1 から TypeScript ソースコードを記述したり、テンプレートを修正してレイアウトやデザインを変更したりすることも可能です。

Claude Code から使用する

Claude Code 等のコーディングエージェントから minitype を使用することもできます。現在の LLM は TypeScript のコード生成能力も高く、minitype も型定義ファイルやドキュメントを提供しているため、minitype を扱うソースコードをコーディングエージェントが生成・編集することも可能であると感じています。

ここでは、minitype とコーディングエージェントを組み合わせた活用例として、**アプリケーションのセキュリティに関するレポートを自動作成**する例をご紹介します。あえて脆弱性を残したモックアプリに関して、以下のプロンプトを Claude Code に与えると、Claude はソースコードを検査した後、脆弱性を発見して、minitype のソースコードを編集してくれました（図 7）。

```
`./app/src/` 以下のコードをセキュリティレビューして、
発見した問題を
`./report-template/security-report
.ts` に反映させた上で、
minitype でセキュリティ評価報告書を生成して
```



図 7: Claude Code による脆弱性検出と minitype ソースコードの編集。

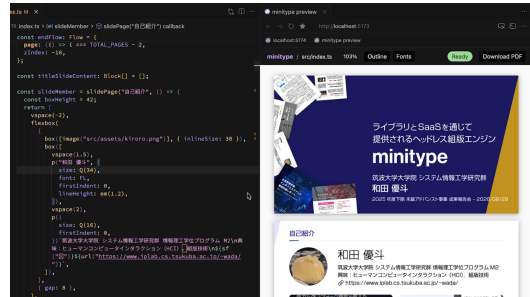


図 9: 成果報告会の発表スライドの作成画面。

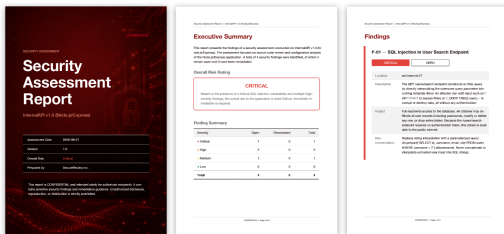


図 8: セキュリティレポートの生成例。

ください。

その後ビルドが実行され、図 8 に示すレポートが生成されました。レポートのテンプレートは事前に用意したのですが、中身に関しては SQL インジェクション等の脆弱性が反映されています。こうしたレポートを手で作成するには時間を要しますが、minitype を利用すれば、一度テンプレートを整備してしまえば、その後はコーディングエージェントを用いて効率的に文書生成することが可能となっています。

多彩な媒体の組版

先日、未踏アドバンスト事業の成果報告会¹⁰があり、組版系のプロジェクトのお約束として、発表スライドも当然 minitype を用いて作成しました (図 9)。

見出しの下のプログレスバー (スライドの進行状況を示す) は自動で挿入されるようになっているほか、スライド中に何度か出現するチェックボックス付きリストは、ヘルパ関数を定義することにより、簡潔に記述できるように工夫しています (図 10)。checkItems は別ファイルに定義されていますが、こうした定型的なヘルパ関数はもはや LLM に書いて

ヘッドレス組版エンジンの特徴

- ✓ 専用言語を使用せずに記述
特定の組版言語 (DSL) に依拠せず TypeScript のオブジェクトとして宣言的にレイアウトを表現
- ✓ ライブラリとして組版機能を提供
データ取得・処理・組版までを同一のコードベース上で実現
➡ アプリケーションと文書生成処理を統合

```
import { box, h2, p, b, minitype } from "minitype/minitype";

const articles = await fetchArticles();
const body = articles.map((a) =>
  box([
    h2([a.title]),
    p(a.body),
  ])
);
await minitype({ body })
  .save("report.pdf");
```

図 10: 上記コードの出力例。フレックスボックスを用いて、左側にリストを、右側にサンプルコードを配置。

もらえば良いものです。

```
checkItems ([
  [
    " 専用言語を使用せずに記述 ",
    [p` 特定の組版言語 (DSL) に依拠せず
\nTypeScript のオブジェクトとして \n 宣言
的にレイアウトを表現 `],
  ],
  [
    " ライブラリとして組版機能を提供 ",
    [
      p` データ取得・処理・組版までを \n 同一
のコードベース上で実現 ` ,
      p` ➡ アプリケーションと \n   文書生成処理
を統合 ` ,
    ],
  ],
])
```

このように minitype では、レポートや書籍等の「いかにも」な書類に限らず、スライドやビジネスレポート、履歴書、

¹⁰ 非公開にて実施。



図 11: ビジネスレポート, 履歴書, 学習教材の生成例. 内容は LLM に資料 (個人ページの URL 等) を与えて生成させたもの.

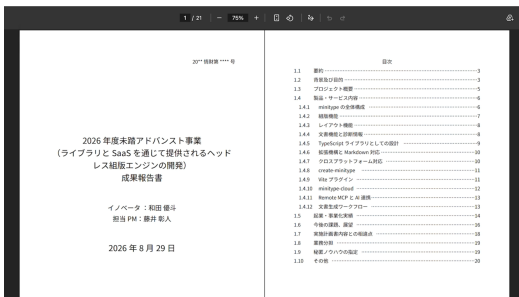


図 12: minitype と LLM を組み合わせて, 所与のフォーマットで 21 ページの成果報告書を生成させた例. デモは大ウケでした.

学習教材等の様々な媒体の組版が可能になっています (図 11). 未踏の成果報告書類もこれで書こうと思っているほどです (図 12).

アプリケーションへの組込み

ここからは, アプリケーションへの minitype の組込みを想定したユースケースを 3 つご紹介します.

旅程表生成アプリケーション

海外渡航時などには旅程表などを印刷して携帯することが多いと思いますが, 旅程表の出力時に minitype による文書生成機能が組込まれていれば, 印刷用にリッチにレイアウトされた文書が生成できるようになります (図 13). 従来, PDF 文書の生成は結構時間の掛かる工程でしたが, minitype を組込むことで, 旅行情報の取得・加工から文書レイアウトまでを同じ TypeScript のコードベース上で扱えるため, 文書生成機能をシームレスに開発することができます.



図 13: 旅程表の生成例.



図 14: 閲覧用の Web ドキュメント.

Web / 印刷用ドキュメントの生成

minitype はサーバサイドでの環境に加えて, ブラウザ上でも動作します. この特徴を活かした例として, minitype の公式ドキュメントのページでは, Web ドキュメントに加えて, PDF 文書を出力できるようになっています. ドキュメント自体は Markdown で管理されており, 単一のソースから閲覧用の Web ドキュメントと, 印刷・保存・配布用の PDF 文書¹¹の両方を生成 (図 14, 15) することにより, 両者を別々に管理する手間を省くことができます.

ブラウザ拡張機能への組込み

さらなる応用として, minitype を埋め込んだブラウザ拡張機能を開発することもできます. この例として, Wikipedia の記事を綺麗に出力・印刷するための拡張機能を考えます.

図 16 に, 未踏事業の実施機関でもある情報処理推進機構 (IPA) に関する Wikipedia 記事の印刷プレビューを示します. ブラウザの標準機能を用いて印刷すると, 余白や段組が印刷向けに最適化されておらず, やや読みにくいレイアウトに仕上がってしまうことが分かります.

¹¹ 配布や長期保存を目的とする場合, やはり表示結果が固定された PDF 文書に依拠してしまう場面は多いと感じます. それが良いかは別問題ではありますが.....

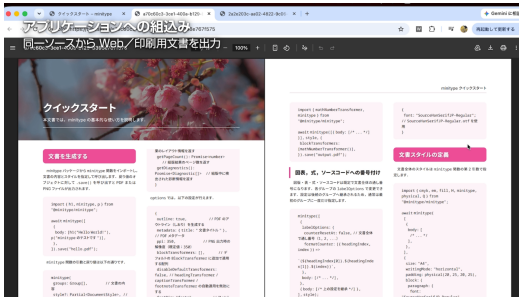


図 15：印刷・配布・保存用の PDF 文書。



図 16：Chrome 標準の印刷プレビュー。



図 17：minitype を組込んだ Chrome 拡張機能による生成例。

そこで、minitype を組込んだ Google Chrome 拡張機能を使用すると、印刷しても見やすい二段組の体裁に変換された PDF 文書を出力することができます（図 17）。この拡張機能では、Document Object Model (DOM) を解析して、HTML 要素を対応する minitype のブロック構造へと変換しています。

ここまで見てきたように、minitype を用いることで、人手での文書作成、LLM を介した文書生成、アプリケーションへの組込みという複数のワークフローにて、様々な種類の文書を同一の組版エンジンから生成することができます。

むすびにかえて

個人的な話になってしまいますが、組版システムの開発は長い期間にわたって取り組んできた、思い入れのある分野です（例 1, 例 2）。2020 年度に未踏 IT 人材発掘・育成事業に採択された際には Twilight という組版処理システムを開発したのですが¹²、力不足により公開まで持って行くことができませんでした。6 年間そのことをずっと後悔していて、今回こうして捲土重来の機会を得て、一般公開まで持って行ったことによって、ようやく肩の荷が降りた気がしています。

未踏事業としては 9 月中旬で一段落ではありますが、開発自体は今後も継続していければと思います。開発や普及活動、商用利用等にご興味のある方はぜひ Twitter（@kyoto_inaniwa）またはメール（me(at)yokohama.dev）までご連絡ください！今後ともどうぞご支援いただけますと幸いです。

¹² 組版に関するプロジェクトはこれまでの未踏でも幾度となく採択され（2007 年度 IdeoType, 2010 年度 evalbook, 2017 年度 SATySFi, 2020 年度 Twilight, 2025 年度 Monk）、曾川 PM も今年 2 月の発表会で「未踏で組版は伝統芸能」と表していた不思議な分野ではあります。